

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO PHÁT TRIỂN ỨNG DỤNG IOT
BÀI THỰC HÀNH 4

Giảng viên : Nguyễn Quốc Uy

Sinh viên : Điền Ngọc Hải

MSV : B22DCPT071

Hà Nội – 2026

Contents

Phần I:	Tổng quan dự án.....	4
1.	Mục tiêu dự án.....	4
2.	Phạm vi hệ thống.....	4
3.	Các yêu cầu chức năng hệ thống.....	4
4.	Yêu cầu về giao diện.....	5
5.	Luồng xử lý dữ liệu.....	5
Phần II:	Thiết kế hệ thống.....	7
1.	Thiết kế giao diện.....	7
1.1.	Dashboard.....	7
1.2.	Sensor History.....	8
1.3.	Device History.....	8
1.4.	Profile.....	9
2.	Luồng hoạt động các chức năng.....	10
2.1.	Đọc cảm biến/thiết bị - Hiển thị data real time.....	10
2.2.	Điều khiển thiết bị.....	12
2.3.	Truy vấn Sensor History.....	14
2.4.	Truy vấn Device History.....	16
Phần III:	Xây dựng hệ thống.....	18
1.	Thiết bị IOT.....	18
2.	MQTT (Mosquitto) Broker.....	18
3.	Thiết kế cơ sở dữ liệu.....	19
3.1.	Sơ đồ lớp.....	19
3.2.	Sơ đồ cơ sở dữ liệu.....	19
4.	API docs:.....	20
5.	Backend.....	23
6.	Frontend.....	23
Phần IV:	Kết quả thực hiện.....	24
1.	Dashboard update và điều khiển thiết bị realtime.....	24
2.	Lịch sử cảm biến (sensor history).....	25
3.	Lịch sử thiết bị (device history).....	26
4.	Profile.....	27
5.	Kết luận.....	27

Hình 1: Dashboard.....	7
Hình 2: Sensor History	8
Hình 3: Device History.....	8
Hình 4: Profile	9
Hình 5: Intinial data realtime sequence diagram.....	10
Hình 6: first intinial sequence diagram	11
Hình 7: device control sequence diagram	12
Hình 8: Hardware/Database Sync sequence diagram.....	13
Hình 9: Sensor history sequence diagram	14
Hình 10: Senson history sequence diagram.....	15
Hình 11: Device History Query sequence diagram	16
Hình 12: Class diagram	19
Hình 13: Database	19
Hình 14: API test 1	20
Hình 15: API test 2	20
Hình 16: API test 3	21
Hình 17: API test 4	22
Hình 18: API test 5	23
Hình 19: Kết quả Dashboard	24
Hình 20: Kết quả Sensor History.....	25
Hình 21: Kết quả Device History	26
Hình 22: Kết quả Profile.....	27

Phần I: Tổng quan dự án

1. Mục tiêu dự án

Xây dựng Hệ thống giám sát môi trường (với 3 yếu tố: Nhiệt độ, độ ẩm và ánh sáng) và điều khiển thiết bị từ xa qua nền tảng Web. Hệ thống được thiết kế theo mô hình Hardware – MQTT – Backend – Frontend, đảm bảo khả năng mở rộng, cập nhật dữ liệu liên tục và điều khiển thiết bị từ xa.

2. Phạm vi hệ thống

Ràng buộc về phần cứng:

- Vi điều khiển trung tâm: ESP8266
- Các cảm biến sử dụng: DHT11 (cho nhiệt độ và độ ẩm); LDR (cho ánh sáng)
- Thiết bị (giả lập): **Quạt, Điều hòa và Đèn** (3 thiết bị kể trên sẽ được giả lập thay thế bằng 3 bóng LED với thứ tự tương ứng: LED 1, 2 và 3)

Ràng buộc về phần mềm:

- Giao thức truyền thông: MQTT (Mosquito broker) cho việc giao tiếp giữa thiết bị IoT và back-end và WebSocket cho việc update realtime data.
- Công nghệ sử dụng cho phát triển: ReactJS, NodeJS, Express và MySQL.

3. Các yêu cầu chức năng hệ thống

Giám sát thời gian thực:

- Yêu cầu: Hệ thống phải thu thập dữ liệu về nhiệt độ, độ ẩm, ánh sáng với chu kỳ 2 giây/lần.
- Hiển thị: Dữ liệu phải được cập nhật tức thời lên Dashboard dưới dạng số liệu và biểu đồ.

Điều khiển thiết bị: Cho phép người dùng bật tắt thiết bị từ giao diện Web.

Quản lý lịch sử thao tác thiết bị: Hệ thống phải lưu trữ toàn bộ giá trị cảm biến và lịch sử thay đổi trạng thái của thiết bị vào trong database. Cho phép người dùng xem, tìm kiếm và sắp xếp dữ liệu theo các tiêu chí như sau: loại cảm biến, giá trị, khoảng thời gian

4. Yêu cầu về giao diện

Giao diện người dùng: Phải bao gồm các trang sau: Dashboard, Sensor History, Device History, Profile. Trong đó:

- Dashboard: Coi như trang home của web, chứa tất cả các thông tin cơ bản liên quan. Ví dụ như tình trạng on/off các thiết bị, dữ liệu realtime thu được từ các sensor, graph về dữ liệu thu được qua thời gian của sensor. Làm mới và cập nhập liên tục từ backend và database (theo chu kỳ 2 giây/lần).
- Sensor History: Trình bày tất cả các dữ liệu thu được từ sensor, được hiển thị ở dạng table (có xử lý phân trang).
- Device History: Trình bày tất cả dữ liệu về lịch sử thay đổi trạng thái của từng thiết bị trong hệ thống ở dạng table (và có xử lý phân trang).
- Profile: Giới hạn lại chỉ bao gồm thông tin người làm dự án (bản thân).

Giao diện truyền thông: Sử dụng giao thức MQTT để publish/subscribe dữ liệu giữa phần cứng và Backend qua các topic tự tạo.

5. Luồng xử lý dữ liệu

Luồng Dữ liệu Cảm biến (Sensor Data Flow)

1. Phần cứng (Hardware): Đọc dữ liệu từ cảm biến định kỳ và đóng gói thành JSON, sau đó publish lên MQTT Broker (Topic: sensor/data).
2. Backend (MQTT Client): Lắng nghe topic sensor/data. Khi có dữ liệu mới, lưu giá trị vào cơ sở dữ liệu MySQL (bảng sensor_data).
3. Backend (WebSocket): Sau khi lưu DB thành công, Backend phát (emit) sự kiện chứa dữ liệu mới qua mạng WebSocket tới tất cả các Frontend đang kết nối mỗi 2 giây.
4. Frontend: Nhận gói tin qua WebSocket, tiến hành cập nhật lại các chỉ số trên thẻ hiển thị (Stat Cards) và vẽ tiếp biểu đồ (Chart.js) mà không cần reload trang.

Luồng Điều khiển Thiết bị (Device Control Flow)

1. Frontend: Người dùng nhấn nút bật/tắt thiết bị trên giao diện. Frontend gọi API POST /api/device/control truyền tên thiết bị và hành động (ON/OFF).
2. Backend (API):

- Tạo một bản ghi hành động mới trong bảng device_action với trạng thái là pending (chờ xử lý).

- Gửi lệnh điều khiển dưới dạng JSON xuống MQTT Broker (Topic: device/control).

- Thiết lập một bộ đếm thời gian (Timeout 6 giây).

3. Phần cứng: Nhận lệnh qua MQTT, thực thi thay đổi vật lý, và publish lại trạng thái thực tế lên MQTT Broker (Topic: device/status).

4. Backend (MQTT Client): Nhận phản hồi từ Hardware. Cập nhật bản ghi pending thành success.

5. Timeout / Rollback: Nếu sau 6 giây Backend không nhận được phản hồi, bộ đếm timeout sẽ kích hoạt, chuyển trạng thái từ pending sang failed, và gửi thông báo lỗi qua WebSocket để Frontend tự động trả (rollback) nút bấm về vị trí thực tế cũ.

Luồng Đồng bộ Hóa Trạng thái (Synchronization Flow)

1. Khi phần cứng bị mất kết nối mạng và kết nối lại, nó sẽ gửi tín hiệu yêu cầu cập nhật trạng thái lên MQTT Broker (Topic: device/sync/request).

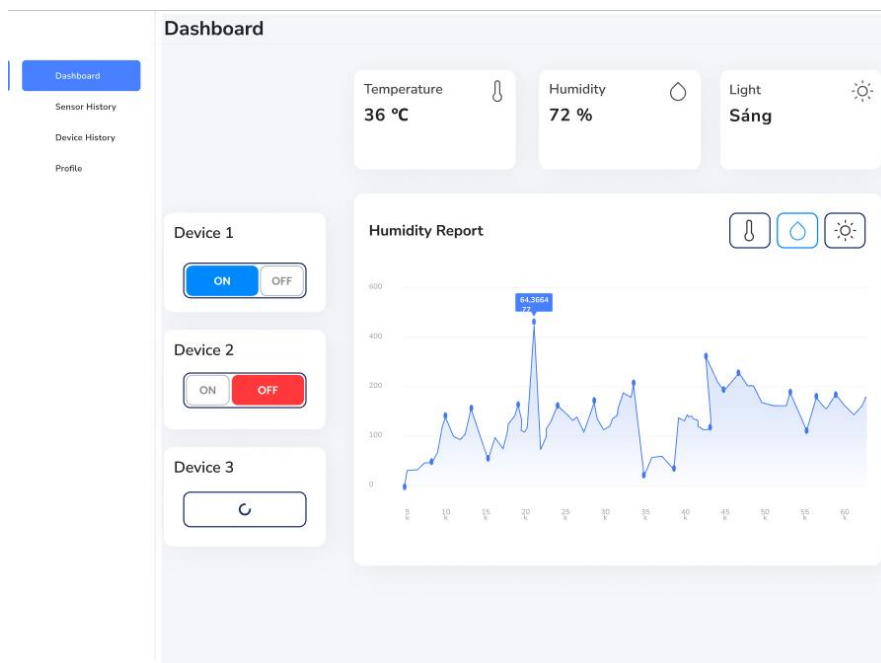
2. Backend nhận được yêu cầu, tiến hành truy xuất vào database để lấy các bản ghi có trạng thái success gần nhất của từng thiết bị (đây là truth source - nguồn sự thật duy nhất).

3. Backend đóng gói toàn bộ trạng thái thực tế của nhà và publish lại xuống phần cứng (Topic: device/sync), giúp phần cứng cập nhật lại chính xác cấu hình trước khi bị mất điện/mất mạng.

Phần II: Thiết kế hệ thống

1. Thiết kế giao diện

1.1. Dashboard

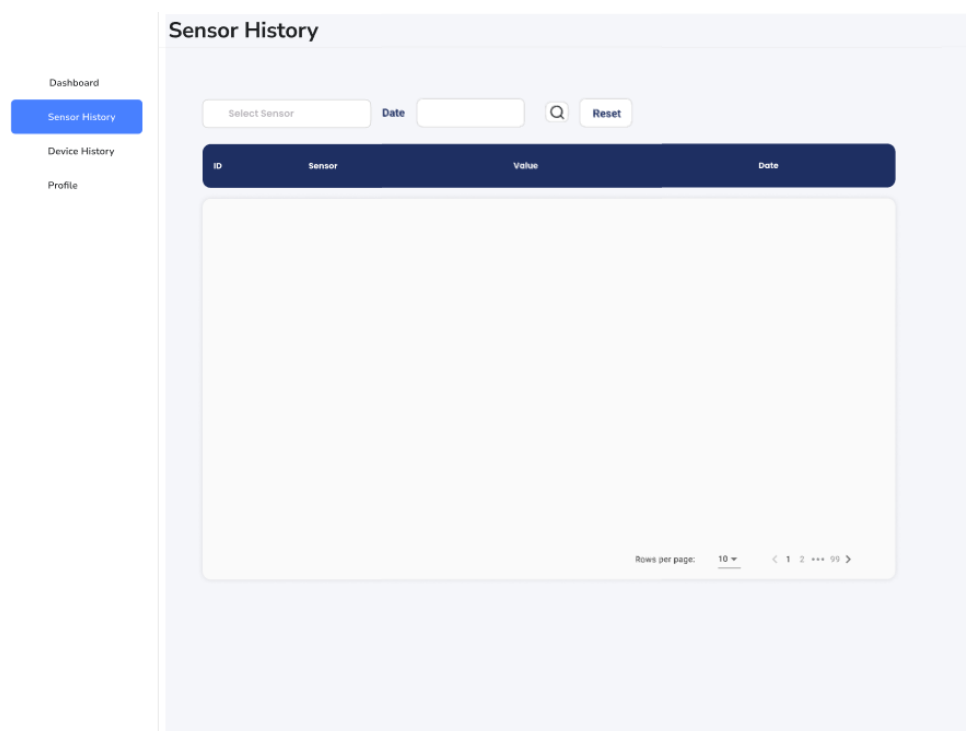


Hình 1: Dashboard

Chức năng Trang Chủ (Dashboard)

- Hiển thị thông số (Stat Cards): Nhận dữ liệu tức thời từ cảm biến.
- Biểu đồ Realtime: Sử dụng Chart.js để vẽ đồ thị đường (Line Chart) 30 mẫu dữ liệu gần nhất của Nhiệt độ, Độ ẩm, và Ánh sáng.
- Công tắc (Toggle Switches): Điều khiển 3 thiết bị Fan, Air Condition, Light.

1.2. Sensor History

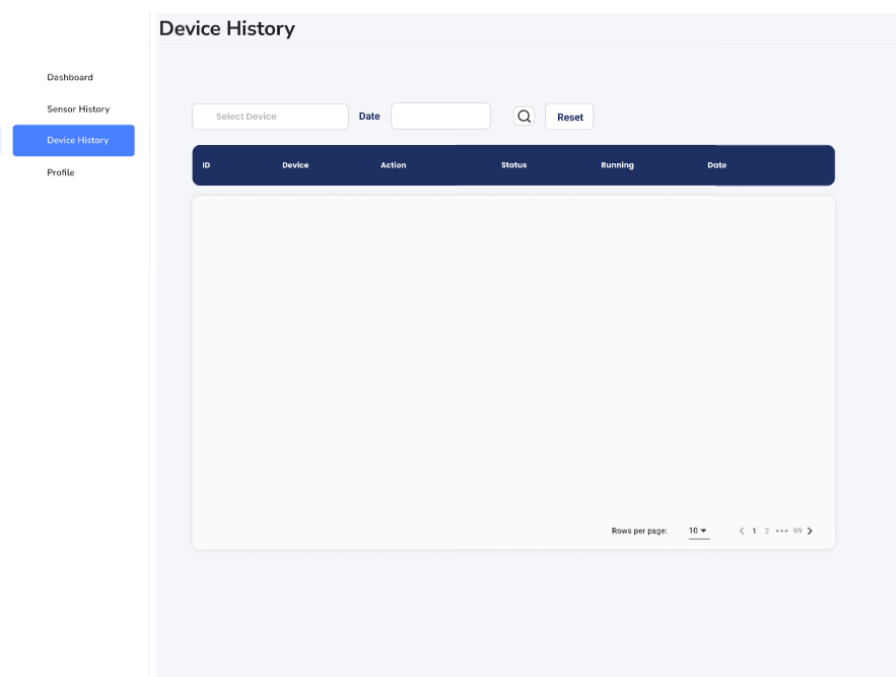


Hình 2: Sensor History

Chức năng: Lịch sử cảm biến

- Hiển thị tất cả dữ liệu cảm biến, có phân trang dữ liệu.
- Tra cứu dữ liệu theo từng cảm biến hoặc theo thời gian

1.3. Device History

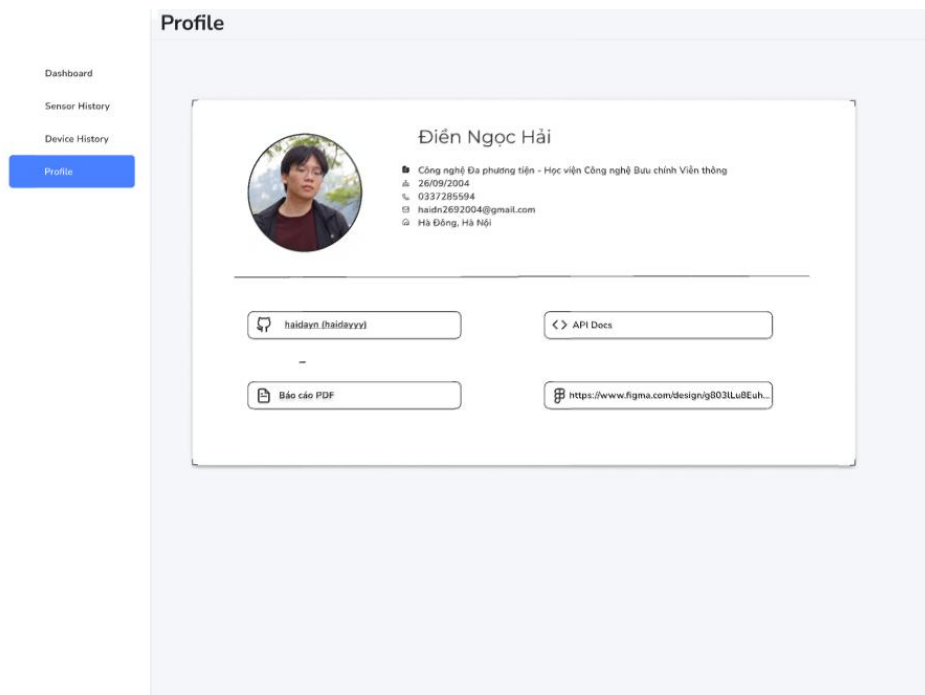


Hình 3: Device History

Chức năng: Lịch sử thiết bị

- Hiển thị tất cả dữ liệu thao tác lên thiết bị, có phân trang dữ liệu.
- Tra cứu theo loại thiết bị, thời gian thao tác thiết bị.

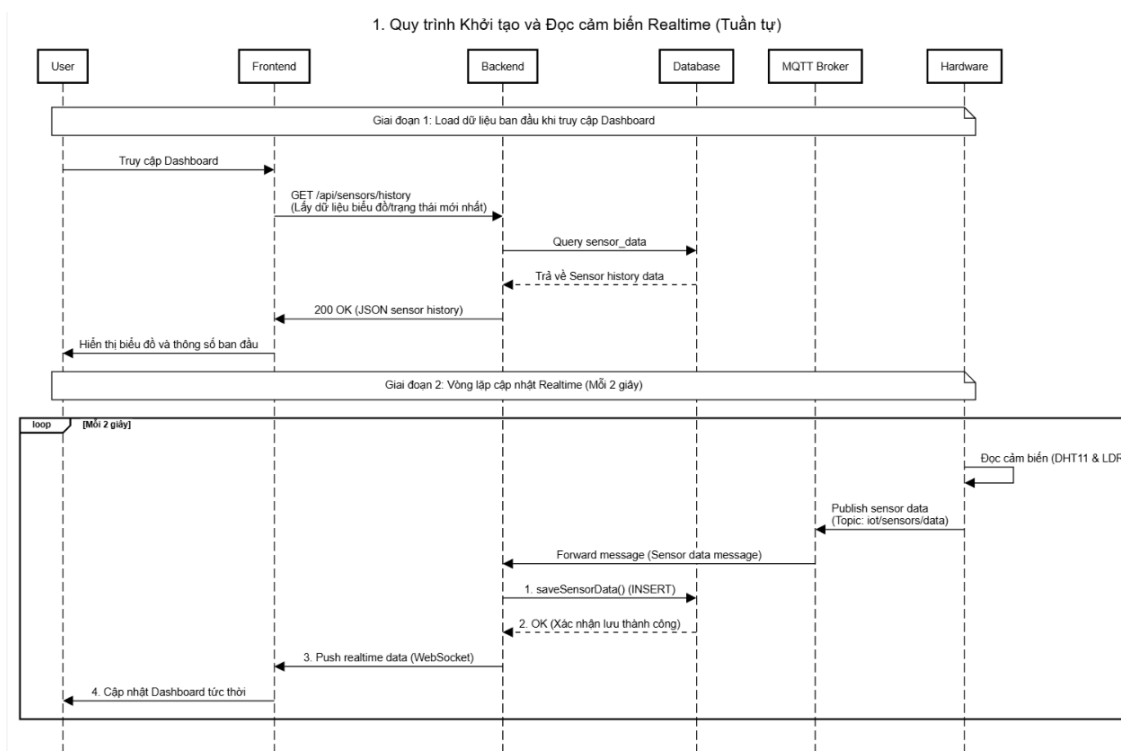
1.4. Profile



Hình 4: Profile

2. Luồng hoạt động các chức năng

2.1. Đọc cảm biến/thiết bị - Hiển thị data real time



Hình 5: Initial data realtime sequence diagram

Luồng tổng quát:

[giai đoạn load data ban đầu]

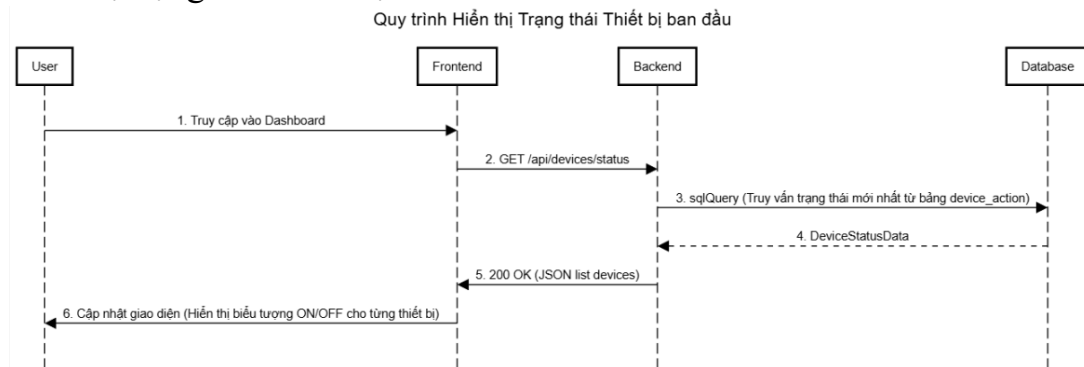
1. Ngay khi truy cập dashboard sẽ gửi yêu cầu GET tới backend API để lấy các giá trị bản ghi cũ.
2. Backend thực hiện truy vấn vào cơ sở dữ liệu MySQL để lấy danh sách lịch sử cảm biến và trạng thái thiết bị gần nhất.
3. Dữ liệu lịch sử sau đó được trả về cho Frontend dưới dạng JSON. Frontend sử dụng dữ liệu này để vẽ biểu đồ và hiển thị các con số trạng thái đầu tiên cho người dùng quan sát.

[giai đoạn loop realtime]

4. Định kỳ 2s/lần hardware sẽ đọc thông số từ cảm biến (DHT11, LDR) để gửi (publish) lên MQTT qua topic iot/sensors/data.
5. MQTT sẽ chuyển tiếp message này đến backend (backend đã subscribe vào topic trên: iot/sensors/data).
6. Backend thực hiện việc lưu data trên database.
7. Khi nhận được phản hồi lưu thành công, backend mới push data lên frontend qua WebSocket.

- Thực hiện việc cập nhật graph và info theo data nhận được từ backend trên frontend - dashboard.

Hiện thị trạng thái thiết bị trên dashboard:

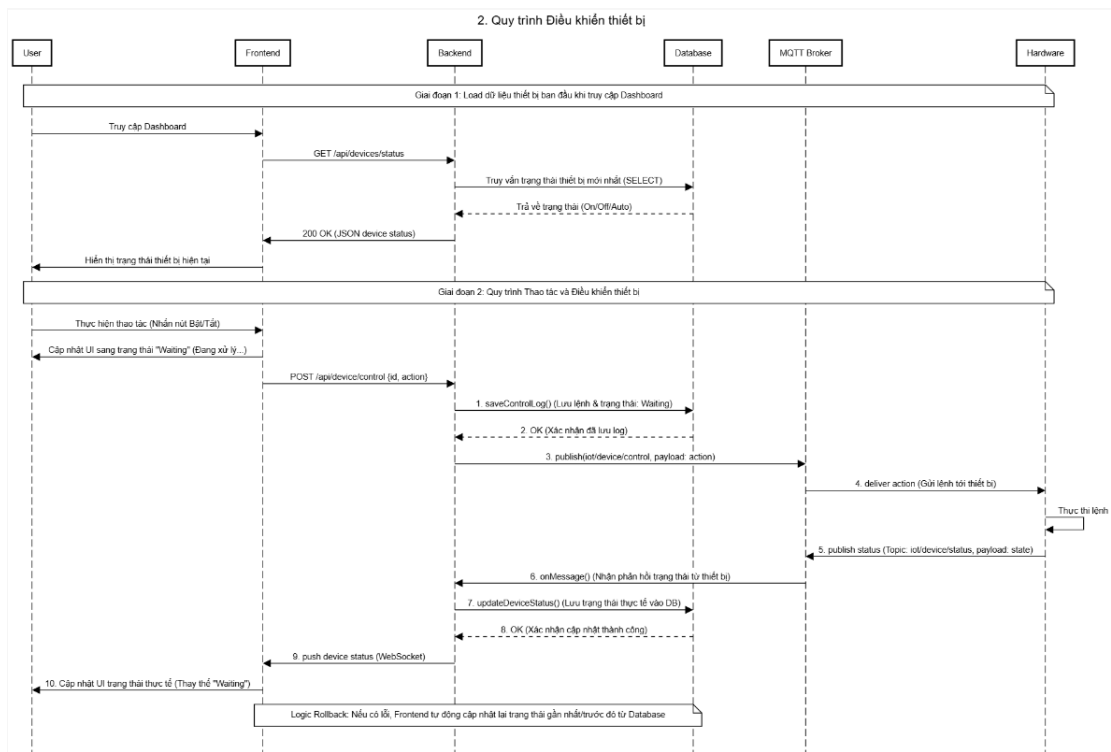


Hình 6: first intinial sequence diagram

Luồng tổng quát:

- Người dùng mở trình duyệt và truy cập vào địa chỉ của trang Dashboard.
- Ngay khi thành phần giao diện (Frontend) được tải, nó sẽ tự động gửi một yêu cầu HTTP GET tới địa chỉ `/api/devices/status` của Backend.
- Backend nhận yêu cầu và chuẩn bị thực hiện truy vấn cơ sở dữ liệu.
- Hệ thống thực hiện một câu lệnh SQL truy vấn vào bảng.
- Cơ sở dữ liệu trả về giá trị thuộc trường **status** cho từng thiết bị.
- Backend đóng gói danh sách trạng thái này thành định dạng JSON và gửi ngược lại cho Frontend.
- Frontend nhận dữ liệu và thay đổi trạng thái hiển thị của các thiết bị.

2.2. Điều khiển thiết bị



Hình 7: device control sequence diagram

Luồng tổng quát:

[giai đoạn load data ban đầu]

1. Khi người dùng bắt đầu truy cập Dashboard, Frontend gửi yêu cầu GET tới Backend để lấy dữ liệu.
2. Backend thực hiện lệnh SELECT để lấy trạng thái mới nhất của các thiết bị (Quạt, Điều hòa, Đèn) từ cơ sở dữ liệu.
3. Dữ liệu được trả về Frontend dưới dạng JSON để giao diện hiển thị đúng trạng thái thực tế ngay từ đầu.

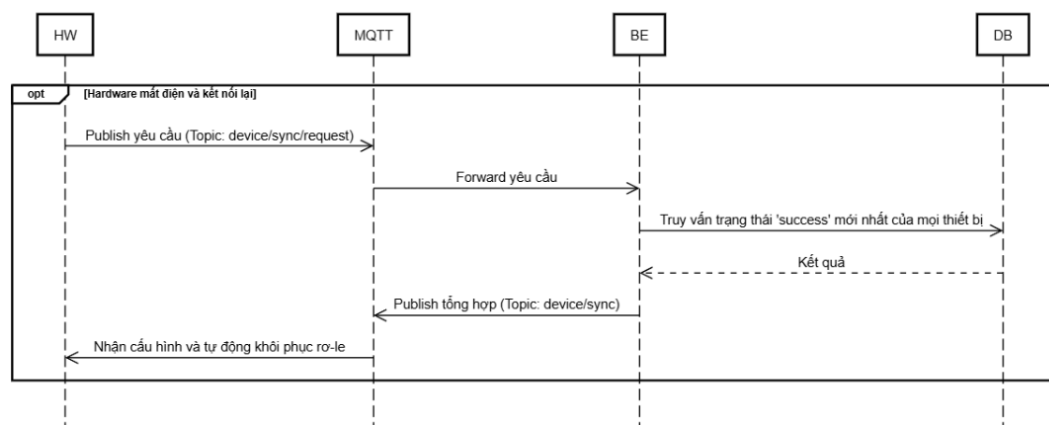
[quy trình thao tác và điều khiển thiết bị]

4. Người dùng thực hiện nhấn nút On/Off hoặc thay đổi giá trị trên giao diện.
5. Frontend lập tức chuyển đổi hiển thị của thiết bị đó sang trạng thái **"Waiting"** (Đang xử lý). Bước này giúp người dùng biết hệ thống đã ghi nhận lệnh và đang trong quá trình giao tiếp với phần cứng.
6. Frontend gửi yêu cầu POST /api/device/control kèm theo định danh thiết bị (deviceId) và hành động mong muốn (action) tới Backend.
7. Backend gọi hàm saveControlLog() để lưu thông tin lệnh điều khiển vào Database với trạng thái ban đầu là "Pending".

8. Chỉ khi Database xác nhận đã lưu log thành công, Backend mới thực hiện publish() lệnh điều khiển lên MQTT Broker tại Topic `iot/device/control`.
9. MQTT Broker chuyển tiếp lệnh tới Hardware. Thiết bị thực hiện thao tác.
10. Sau khi thực hiện, Hardware gửi bản tin phản hồi trạng thái thực tế (`deviceState`) lên Topic `iot/device/status` thông qua MQTT Broker.
11. Backend nhận phản hồi từ Hardware, gọi hàm `updateDeviceStatus()` để ghi đè trạng thái thực tế vào Database.
12. Sau khi Database cập nhật thành công, Backend đẩy dữ liệu qua WebSocket tới Frontend.
13. Frontend nhận dữ liệu, thay thế trạng thái "Waiting" bằng trạng thái thực tế (ví dụ: chuyển từ "Waiting" sang "On").

Cơ chế Rollback: Trong trường hợp xảy ra lỗi (mất kết nối, thiết bị không phản hồi), Frontend sẽ tự động truy vấn lại trạng thái hợp lệ gần nhất trong Database để cập nhật lại giao diện, tránh tình trạng hiển thị sai lệch so với thực tế.

Đồng bộ trạng thái Hardware/Database:



Hình 8: Hardware/Database Sync sequence diagram

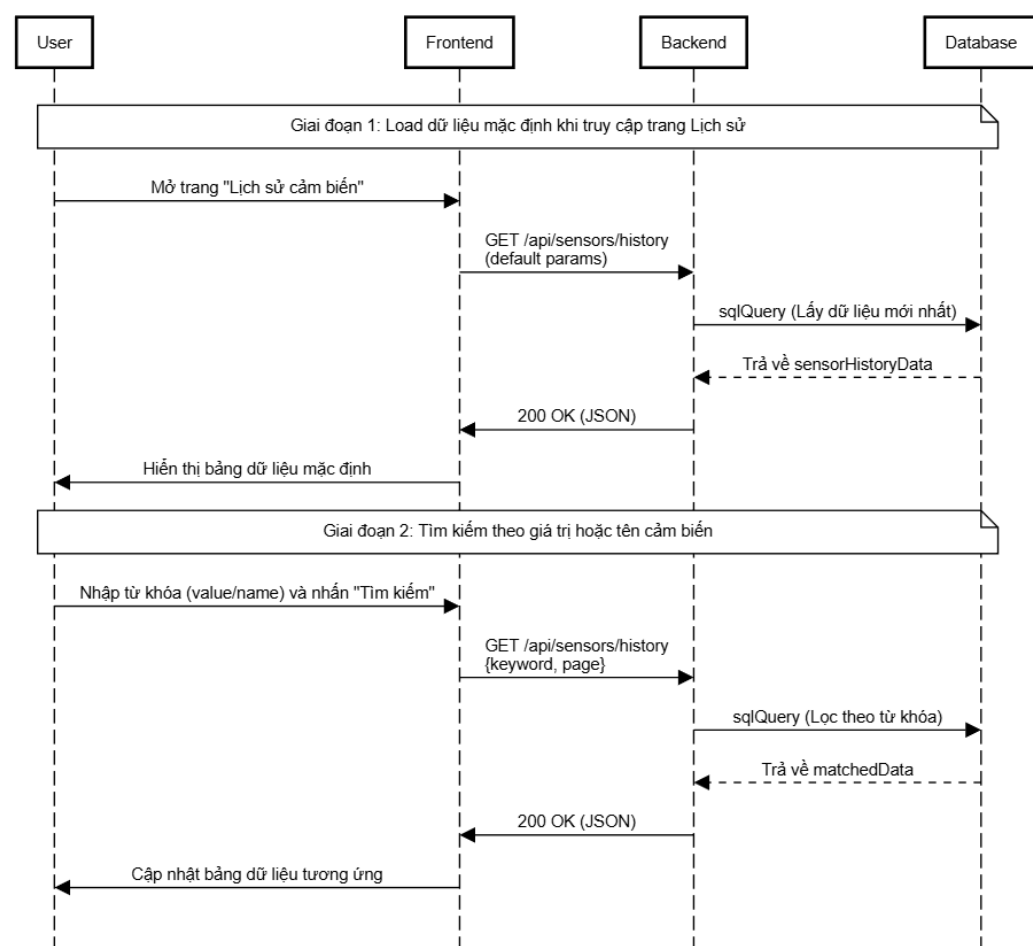
Luồng tổng quát:

1. Khi thiết bị phần cứng (Hardware) gặp sự cố mất điện và vừa thiết lập lại được kết nối, nó sẽ tự động gửi (Publish) một bản tin yêu cầu đồng bộ trạng thái tới MQTT Broker thông qua chủ đề `device/sync/request`.
2. MQTT Broker nhận được thông điệp và ngay lập tức chuyển tiếp (Forward) yêu cầu này tới hệ thống Backend đang lắng nghe.
3. Backend tiếp nhận yêu cầu đồng bộ và tiến hành thực hiện một lệnh truy vấn vào Cơ sở dữ liệu (Database).

- Hệ thống tìm kiếm và lấy ra trạng thái hoạt động thành công ('success') được lưu lại gần nhất của tất cả các thiết bị.
- Cơ sở dữ liệu trả về tập kết quả chứa các trạng thái này cho Backend.
- Backend đóng gói, tổng hợp lại các dữ liệu trạng thái và gửi (Publish) bản tin cấu hình này ngược lại MQTT Broker thông qua chủ đề device/sync.
- Phần cứng (Hardware) nhận được bản tin cấu hình từ MQTT Broker, sau đó tự động đọc dữ liệu và điều chỉnh để khôi phục lại trạng thái của các relay khớp với dữ liệu trên hệ thống.

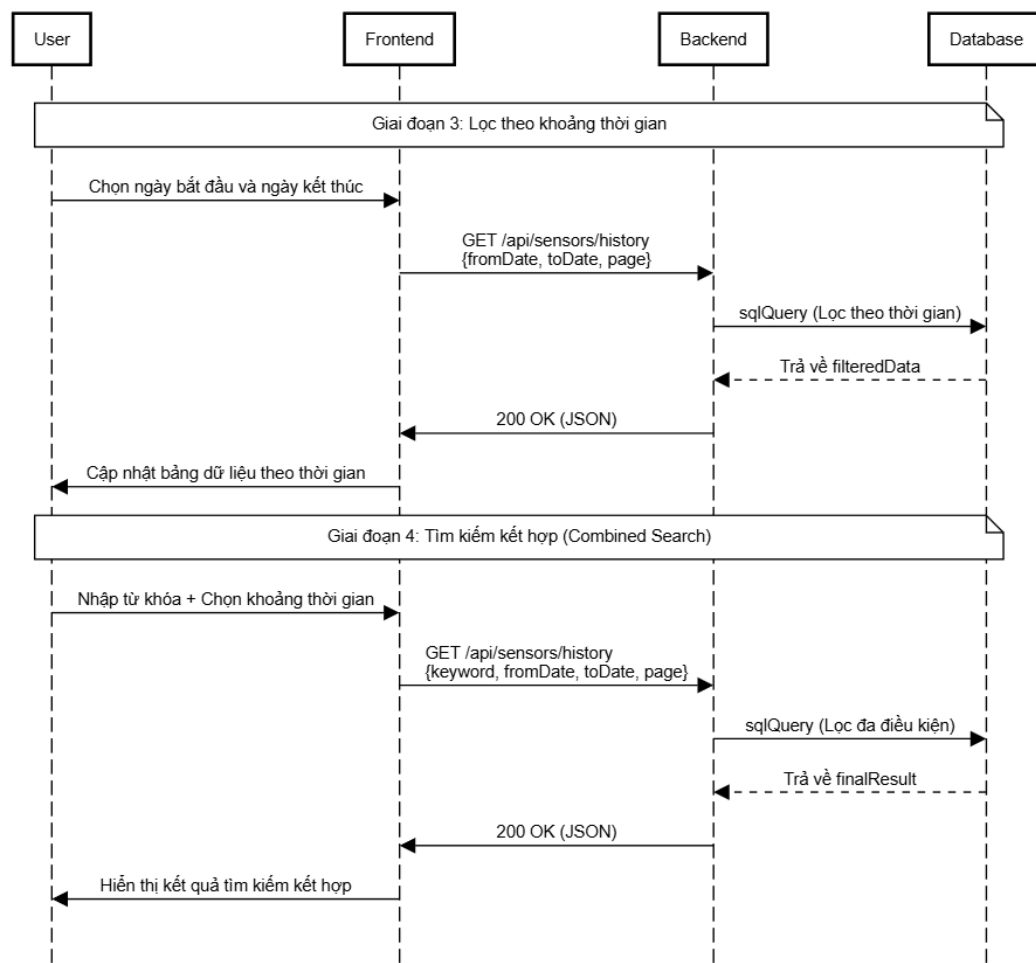
2.3. Truy vấn Sensor History

3. Lịch sử cảm biến



Hình 9: Sensor history sequence diagram

3. Lịch sử cảm biến



Hình 10: Sensor history sequence diagram

Luồng tổng quát

[giai đoạn load data ban đầu]

1. Khi người dùng mở trang Sensor History - "Lịch sử cảm biến", **Frontend** gửi yêu cầu GET /api/sensors/history với các tham số mặc định (thường là trang 1 và không có bộ lọc).
2. **Backend** nhận yêu cầu và thực thi sqlQuery để lấy danh sách các bản ghi mới nhất từ Database.
3. Dữ liệu lịch sử được trả về và hiển thị ngay lập tức lên bảng giúp người dùng có thông tin tổng quan mà không cần thao tác thêm.

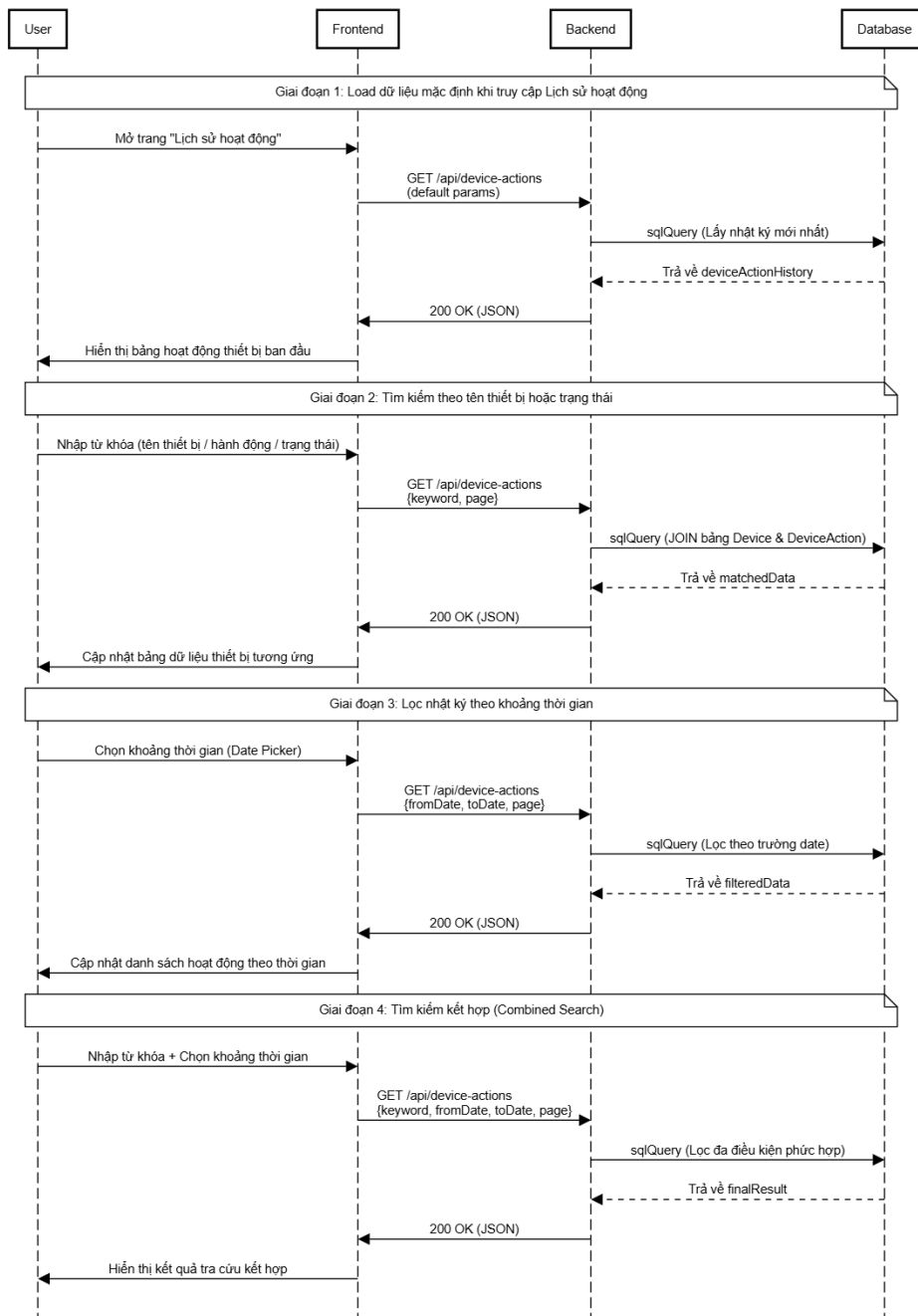
[thao tác lọc data riêng biệt]

4. User nhập/chọn từ khóa (ví dụ: chọn temperature hoặc giá trị "27") hoặc nhập ngày tháng vào ô tìm kiếm.
5. Hệ thống thực thi sqlQuery (tổng quan) với điều kiện lọc để trích xuất các dữ liệu được ghi nhận trong khoảng đó.

6. Kết quả trả về từ sqlQuery không chỉ bao gồm danh sách dữ liệu (data) mà còn có tổng số bản ghi (total) để phục vụ tính năng phân trang.
7. **Frontend** nhận phản hồi JSON, cập nhật lại trạng thái của bảng và render các dòng dữ liệu mới tương ứng với yêu cầu lọc của người dùng.

2.4. Truy vấn Device History

4. Quy trình Truy vấn Lịch sử thiết bị (Phân đoạn Filter)



Hình 11: Device History Query sequence diagram

Luồng tổng quát:

[giai đoạn load data ban đầu]

1. Khi người dùng mở trang Device History - "Lịch sử thiết bị", **Frontend** gửi yêu cầu GET /api/device-actions với các tham số mặc định (ví dụ: lấy 10 bản ghi mới nhất).
2. **Backend** nhận yêu cầu và thực thi sqlQuery để lấy danh sách các bản ghi mới nhất từ Database.
3. Dữ liệu được trả về và hiển thị lên bảng nhật ký, giúp người dùng thấy ngay các hoạt động gần đây của Quạt, Điều hòa hoặc Đèn.

[thao tác lọc data riêng biệt]

4. User nhập/chọn từ hoặc trạng thái hoạt động (running/none) hoặc loại hành động (on/off) hoặc nhập ngày tháng vào ô tìm kiếm.
5. Hệ thống thực thi sqlQuery (tổng quan) với điều kiện lọc để trích xuất các dữ liệu được ghi nhận trong khoảng đó.
6. Kết quả trả về từ sqlQuery không chỉ bao gồm danh sách dữ liệu (data) mà còn có tổng số bản ghi (total) để phục vụ tính năng phân trang.
7. **Frontend** nhận phản hồi JSON, cập nhật lại trạng thái của bảng và render các dòng dữ liệu mới tương ứng với yêu cầu lọc của người dùng.

Phần III: Xây dựng hệ thống

1. Thiết bị IOT

ESP8266 Là thành phần trung tâm, chịu trách nhiệm cho việc: điều khiển các thiết bị, thu thập dữ liệu từ cảm biến, giao tiếp với các tác nhân bên ngoài (MQTT).

Chức năng chính:

- Đọc dữ liệu từ các cảm biến như: DHT11 (độ ẩm, nhiệt độ), LDR (ánh sáng)
- Gửi tín hiệu lên MQTT qua topic, tín hiệu bao gồm dữ liệu cảm biến và các phản hồi tới backend
- Nhận lệnh điều khiển từ backend qua MQTT

2. MQTT (Mosquitto) Broker

Hệ thống sử dụng MQTT Broker (Mosquitto) làm trung gian truyền dữ liệu giữa thiết bị IoT và Backend.

Vai trò:

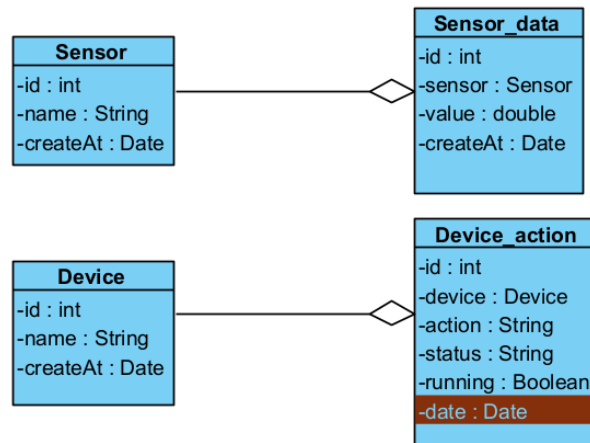
- Nhận dữ liệu gửi đến từ thiết bị IoT
- Chuyển dữ liệu đến backend
- Nhận lệnh điều khiển thiết bị từ backend

Các topic sử dụng:

- TOPIC: `iot/sensor/data`: Kênh hardware gửi dữ liệu cảm biến
- TOPIC: `iot/device/control`: Kênh gửi yêu cầu điều khiển tới hardware
- TOPIC: `iot/device/status`: Kênh phản hồi của hardware đối với lệnh điều khiển từ backend
- TOPIC: `iot/device/sync`: Kênh đồng bộ trạng thái giữa hardware và database

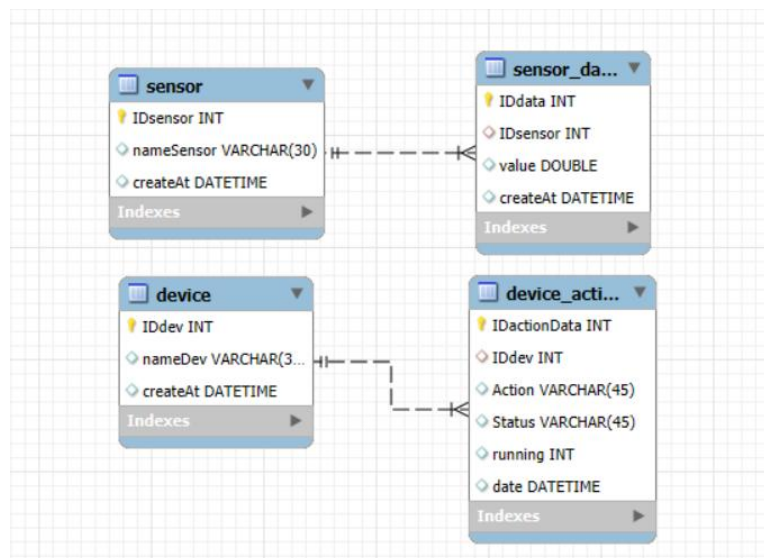
3. Thiết kế cơ sở dữ liệu

3.1. Sơ đồ lớp



Hình 12: Class diagram

3.2. Sơ đồ cơ sở dữ liệu

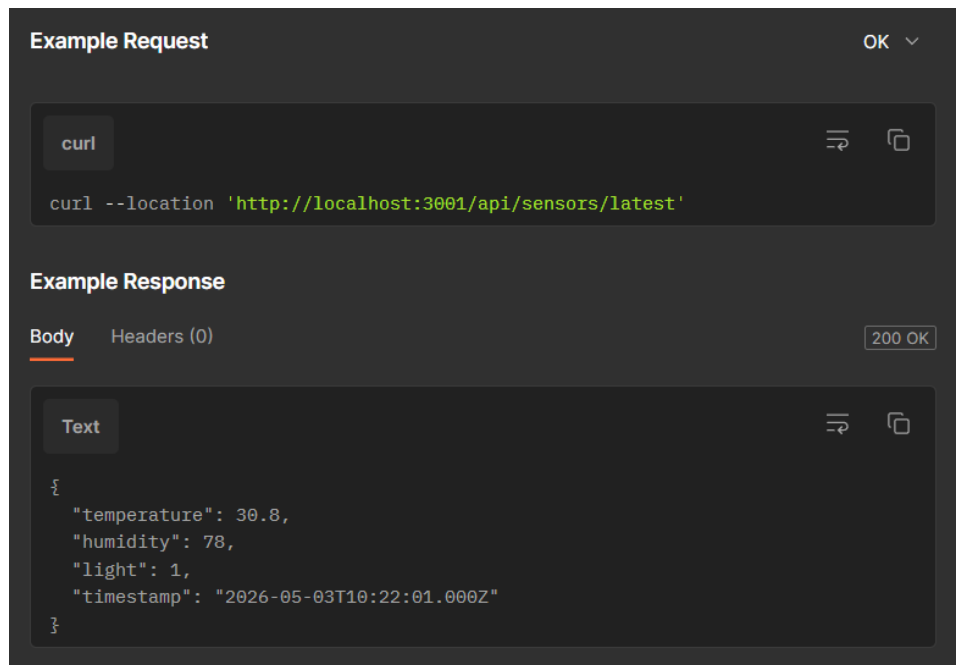


Hình 13: Database

4. API docs:

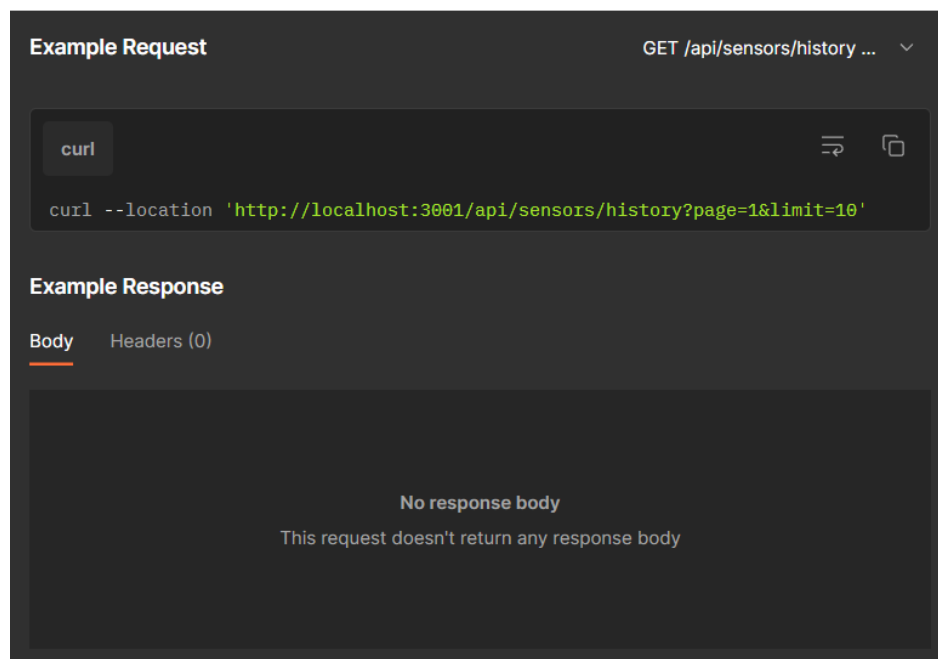
Cảm biến (Sensor)

- GET /api/sensors/latest: Lấy 1 bản ghi mới nhất của từng cảm biến để hiển thị Dashboard (Nhiệt độ, Độ ẩm, Ánh sáng).



Hình 14: API test 1

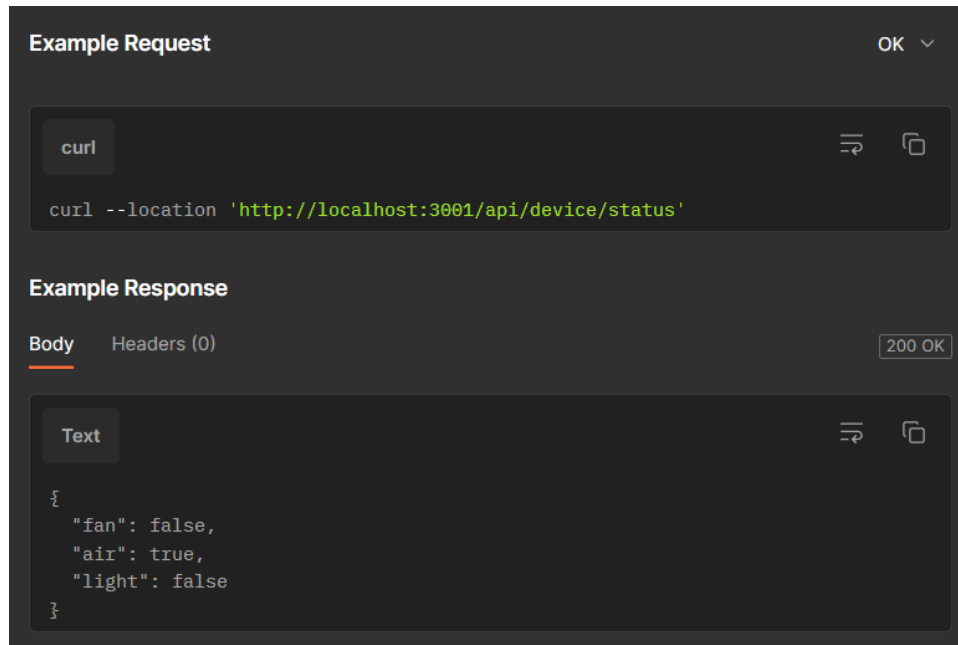
- GET /api/sensors/history: Truy xuất lịch sử dữ liệu cảm biến; Hỗ trợ: Phân trang (page/limit), Lọc theo tên cảm biến, Lọc thời gian linh hoạt (exactDate), Tìm kiếm giá trị (keyword).



Hình 15: API test 2

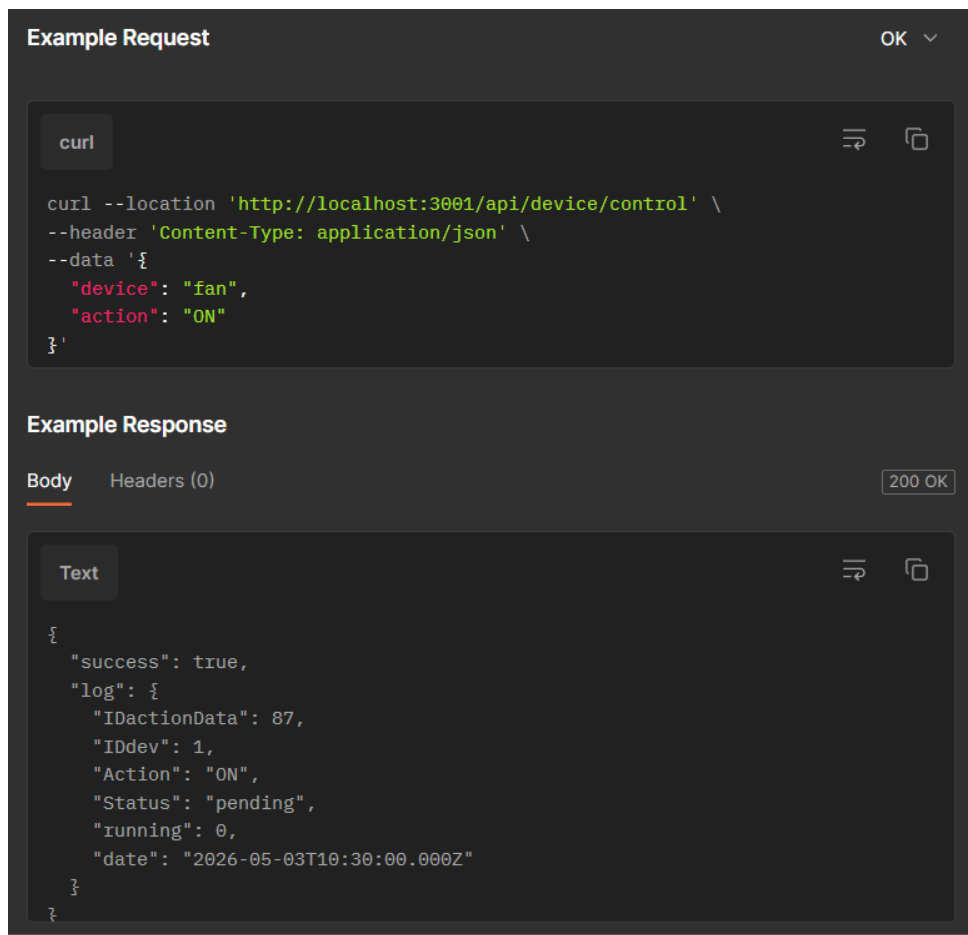
Thiết bị (Device)

- GET /api/device/status: Lấy trạng thái ON/OFF hiện tại của tất cả thiết bị (Quạt, Điều hòa, Đèn) dựa trên bản ghi thành công gần nhất.



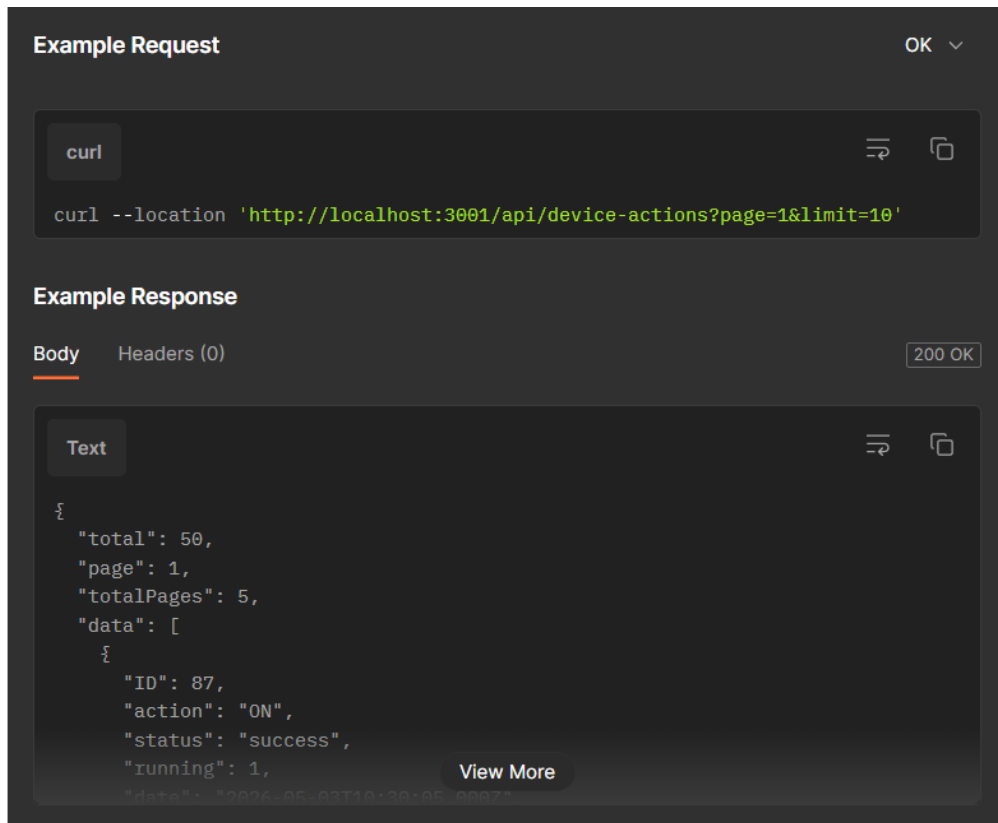
Hình 16: API test 3

- POST /api/device/control: Gửi lệnh điều khiển thiết bị (ON/OFF); Tích hợp Timeout 6 giây (tự động mark failed & rollback UI nếu phần cứng không phản hồi).



Hình 17: API test 4

- GET /api/device-actions: Truy xuất lịch sử điều khiển thiết bị; Hỗ trợ: Phân trang, Lọc trạng thái (success/failed/pending), Lọc thời gian linh hoạt (exactDate).



Hình 18: API test 5

5. Backend

Công nghệ sử dụng:

- Môi trường chạy: Node.js.
- Web Framework: Express.js (Xử lý RESTful API và Middleware).
- Cơ sở dữ liệu: MySQL (Lưu trữ lịch sử, cấu hình).
- ORM (Object-Relational Mapping): Sequelize (Tương tác với DB qua code JS, tránh viết SQL thuần, giúp bảo mật và dễ bảo trì).
- Giao thức Realtime: Socket.IO (Bắn dữ liệu từ Server lên Web), MQTT.js (Giao tiếp với phần cứng).

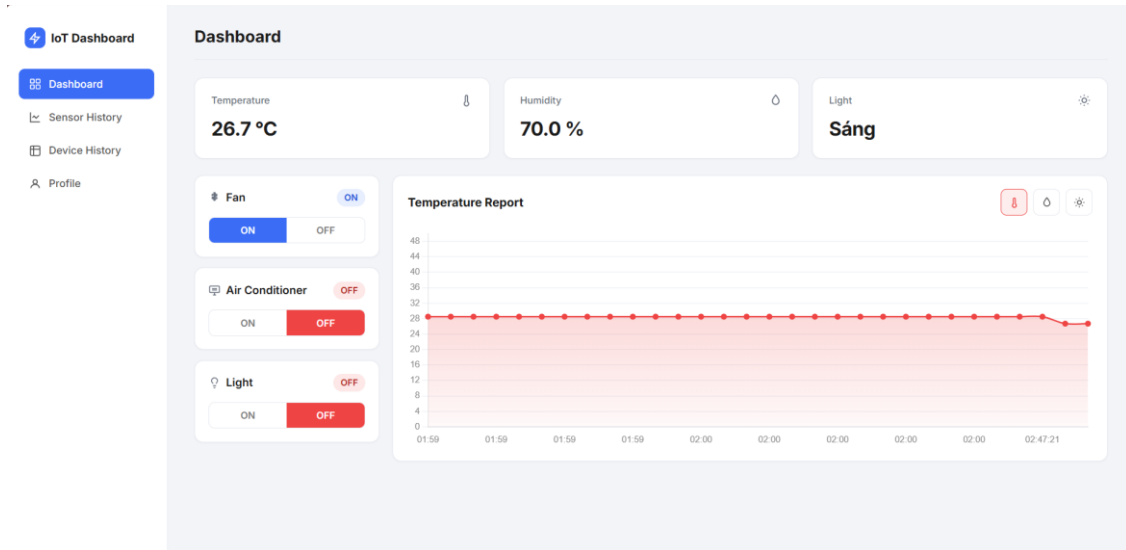
6. Frontend

Công nghệ sử dụng:

- Library: React.js (Xây dựng giao diện hướng Component).
- Build Tool: Vite (Đóng gói nhanh và tối ưu hóa module).
- HTTP Client: Axios (Gọi REST API).
- Biểu đồ (Charts): Chart.js & React-Chartjs-2.
- Giao thức Realtime: Socket.IO-client.
- CSS: Vanilla CSS

Phần IV: Kết quả thực hiện

1. Dashboard update và điều khiển thiết bị realtime



Hình 19: Kết quả Dashboard

Dashboard cung cấp giao diện trực quan giúp người dùng theo dõi dữ liệu môi trường theo thời gian thực, bao gồm:

- Nhiệt độ (°C)
- Độ ẩm (%)
- Mức sáng

Dữ liệu được cập nhật liên tục mỗi 2 giây từ thiết bị phần cứng (ESP8266) thông qua giao thức MQTT và WebSocket.

Người dùng có thể điều khiển 3 thiết bị:

- Quạt (Device 1)
- Điều hòa (Device 2)
- Đèn (Device 3)

2. Lịch sử cảm biến (sensor history)

ID	Sensor	Value	Date
70355	Humidity	70.0 %	2026-05-04 03:50:19
70354	Temperature	26.7 °C	2026-05-04 03:50:19
70356	Light	0.0 lux	2026-05-04 03:50:19
70352	Humidity	70.0 %	2026-05-04 03:50:17
70351	Temperature	26.7 °C	2026-05-04 03:50:17
70353	Light	0.0 lux	2026-05-04 03:50:17
70348	Temperature	26.7 °C	2026-05-04 03:50:15
70349	Humidity	70.0 %	2026-05-04 03:50:15
70350	Light	0.0 lux	2026-05-04 03:50:15
70345	Temperature	26.7 °C	2026-05-04 03:50:13

Hình 20: Kết quả Sensor History

Dữ liệu lấy từ bảng sensor_data.

Chức năng:

- Tìm kiếm theo:
 - Thiết bị
 - Giá trị
 - Thời gian
- Sắp xếp theo:
 - Thời gian
 - Giá trị
- Phân trang dữ liệu

3. Lịch sử thiết bị (device history)

ID	Device	Action	Status	Running	Date
328	Fan	Turn OFF	Success	—	2026-05-04 03:01:34
327	Fan	Turn ON	Success	Running	2026-05-04 02:47:23
326	Light	Turn OFF	Success	—	2026-05-04 01:10:32
325	Fan	Turn OFF	Success	—	2026-05-04 01:10:31
324	Light	Turn OFF	Failed	—	2026-05-04 01:09:51
323	Air	Turn ON	Failed	—	2026-05-04 01:09:50
322	Fan	Turn OFF	Failed	—	2026-05-04 01:09:49
321	Fan	Turn ON	Success	Running	2026-05-04 01:09:42
320	Air	Turn OFF	Success	—	2026-05-04 01:09:41
319	Light	Turn ON	Success	Running	2026-05-04 01:09:38

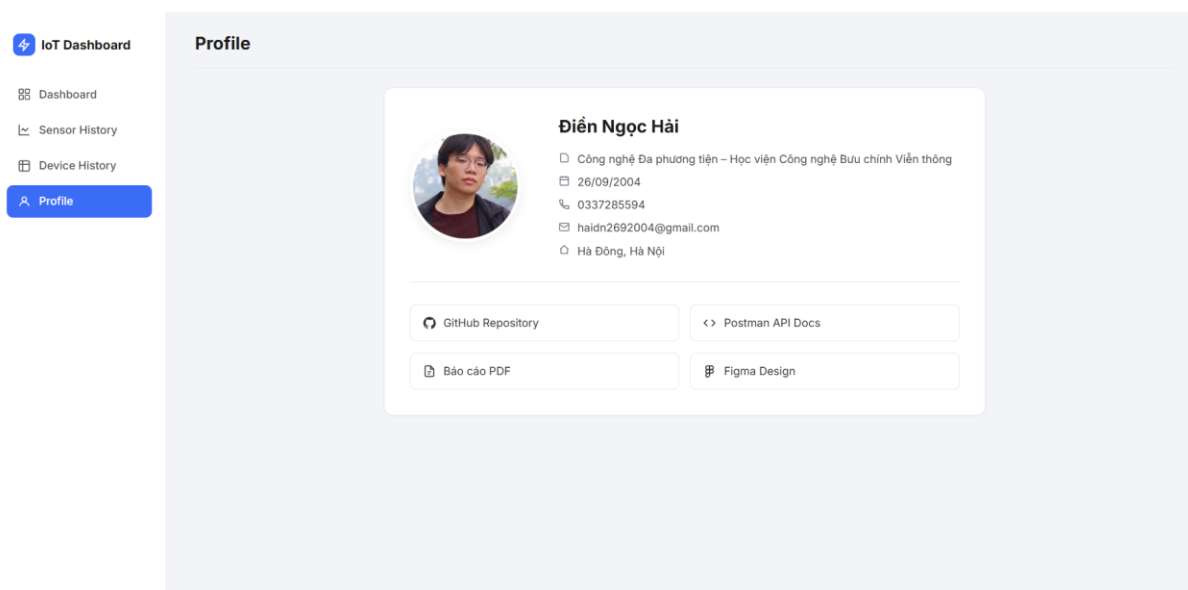
Hình 21: Kết quả Device History

Dữ liệu lấy từ bảng device_action.

Chức năng:

- Tìm kiếm theo:
 - Thiết bị
 - Trạng thái
 - Thời gian
- Phân trang dữ liệu

4. Profile



Hình 22: Kết quả Profile

Trang thông tin hiển thị các thông tin như sau:

- Thông tin sinh viên.
- Hyperlink để đến repo Github của Project, Figma, API doc và Báo cáo (PDF).

5. Kết luận

Hệ thống IoT giám sát môi trường và điều khiển thiết bị đã được xây dựng thành công với đầy đủ các chức năng như sau:

- Giám sát dữ liệu môi trường theo thời gian thực
- Điều khiển thiết bị từ xa thông qua Web
- Lưu trữ và truy xuất dữ liệu lịch sử

Hệ thống đảm bảo các yêu cầu sau:

- Tính chính xác trong thu thập và xử lý dữ liệu
- Tính ổn định khi vận hành liên tục
- Khả năng mở rộng trong tương lai

Thông qua quá trình triển khai, hệ thống đã chứng minh được tính khả thi trong việc áp dụng vào các bài toán thực tế như:

- Nhà thông minh (Smart Home)
- Giám sát môi trường